C8 / SQL QUERY 01

Joined allocation details

Executed against synclab_demonstration.db with fictional sample data. Full output is also supplied as CSV and JSON.

```
-- Query 1: Joined allocation details with number of normalized child slots.
SELECT r.id, r.category AS workshop, s.name AS student,
       r.record_date AS allocation_date, r.project_name,
       r.status, COUNT(bs.time_slot) AS selected_slots
FROM records AS r
JOIN students AS s ON s.index_no = r.index_no
JOIN workshops AS w ON w.category = r.category
LEFT JOIN booking_slots AS bs ON bs.record_id = r.id
GROUP BY r.id, r.category, s.name, r.record_date, r.project_name, r.status
ORDER BY r.record_date, r.id;
```

Printed columns: booking ID, workshop, allocation date, status and number of slots. The full export also includes student and project fields.

Observed result: 8 rows

ID	Workshop	Date	Status	Slots
1	Computer	2026-10-20	Pending	1
2	Computer	2026-10-20	Approve	1
3	Computer	2026-10-20	Reject	1
4	Electronics	2026-10-21	Approve	2
5	Concrete	2026-10-22	Pending	2
7	Geotechnical	2026-10-22	Approve	1
6	Manufacturing	2026-10-23	Reject	2
8	Measurement Laboratory	2026-10-24	Pending	1

Joins reconstruct allocation details while COUNT reports atomic child slots. Eight booking rows remain eight rows even where two slots were selected.

Reproduce: create an empty database, execute create_tables.sql, then sample_data.sql, then 01_allocation_details.sql. Query outputs describe the demonstration data, not live university usage.

C8 / SQL QUERY 02

Workshop utilization, including unused workshops

Executed against synclab_demonstration.db with fictional sample data. Full output is also supplied as CSV and JSON.

```
-- Query 2: Include unused workshops; Reject allocations do not reserve slots.
SELECT w.category AS workshop,
       COUNT(DISTINCT r.id) AS allocations,
       COUNT(DISTINCT CASE WHEN r.status <> 'Reject' THEN r.id END) AS active_allocations,
       SUM(CASE WHEN r.status <> 'Reject' AND bs.time_slot IS NOT NULL
                THEN 1 ELSE 0 END) AS reserved_slots
FROM workshops AS w
LEFT JOIN records AS r ON r.category = w.category
LEFT JOIN booking_slots AS bs ON bs.record_id = r.id
GROUP BY w.category
ORDER BY w.category;
```

Observed result: 10 rows

Workshop	Bookings	Active bookings	Reserved slots
Biochemistry	0	0	0
Computer	3	2	2
Concrete	1	1	2
Drawing Office	0	0	0
Electrical Machines & Communications Laboratory	0	0	0
Electronics	1	1	2
Geotechnical	1	1	1
Manufacturing	1	0	0
Measurement Laboratory	1	1	1
Surveying	0	0	0

LEFT JOIN includes unused workshops. Computer has three records but only two active reservations. Rejected Manufacturing records reserve no slots.

Reproduce: create an empty database, execute create_tables.sql, then sample_data.sql, then 02_workshop_utilization.sql. Query outputs describe the demonstration data, not live university usage.

C8 / SQL QUERY 03

Available Computer Lab slots on 20 October 2026

Executed against synclab_demonstration.db with fictional sample data. Full output is also supplied as CSV and JSON.

```
-- Query 3: Free Computer Lab slots for 20 October 2026.
-- The rejected record at 13:30 does not block that slot.
WITH daily_slots(slot_order, time_slot) AS (
    VALUES (1, '8.30 am - 10.30 am'),
           (2, '10.30 am - 12.30 pm'),
           (3, '1.30 pm - 3.30 pm'),
           (4, '3.30 pm - 5.30 pm')
)
SELECT 'Computer' AS workshop, '2026-10-20' AS allocation_date,
       ds.time_slot AS available_time_slot
FROM daily_slots AS ds
WHERE NOT EXISTS (
    SELECT 1
    FROM records AS r
    JOIN booking_slots AS bs ON bs.record_id = r.id
    WHERE r.category = 'Computer'
      AND r.record_date = '2026-10-20'
      AND r.status <> 'Reject'
      AND bs.time_slot = ds.time_slot
)
ORDER BY ds.slot_order;
```

Observed result: 2 rows

Workshop	Date	Available slot
Computer	2026-10-20	1.30 pm - 3.30 pm
Computer	2026-10-20	3.30 pm - 5.30 pm

The two afternoon Computer slots remain available. A rejected record at 1.30 pm does not block that slot.

Reproduce: create an empty database, execute create_tables.sql, then sample_data.sql, then 03_available_slots.sql. Query outputs describe the demonstration data, not live university usage.

C8 / SQL QUERY 04

Review workload and slot commitments by status

Executed against synclab_demonstration.db with fictional sample data. Full output is also supplied as CSV and JSON.

```
-- Query 4: Administrator review workload and slot commitments by status.
SELECT r.status, COUNT(DISTINCT r.id) AS allocations,
       COUNT(bs.time_slot) AS selected_slots,
       SUM(CASE WHEN r.status <> 'Reject' THEN 1 ELSE 0 END) AS reserved_slots
FROM records AS r
JOIN booking_slots AS bs ON bs.record_id = r.id
GROUP BY r.status
ORDER BY CASE r.status WHEN 'Pending' THEN 1
                       WHEN 'Approve' THEN 2
                       WHEN 'Reject' THEN 3 END;
```

Observed result: 3 rows

Status	Bookings	Slots	Reserved slots
Pending	3	4	4
Approve	3	4	4
Reject	2	3	0

Three Pending and three Approve bookings each reserve four slots. Two Reject bookings have three selected historical slots and reserve zero.

Reproduce: create an empty database, execute create_tables.sql, then sample_data.sql, then 04_status_workload.sql. Query outputs describe the demonstration data, not live university usage.

C8 / SQL QUERY 05

Student allocation history across workshops

Executed against synclab_demonstration.db with fictional sample data. Full output is also supplied as CSV and JSON.

```
-- Query 5: One student can hold allocations at different workshops.
SELECT s.index_no, s.name AS student, r.category AS workshop,
       r.record_date AS allocation_date, r.project_name, r.status
FROM students AS s
JOIN records AS r ON r.index_no = s.index_no
WHERE s.index_no = 'DEMO/NT/001'
ORDER BY r.record_date, r.id;
```

Filter: DEMO/NT/001 (Demo Student A). The full export retains student identity columns.

Observed result: 3 rows

Workshop	Date	Project	Status
Computer	2026-10-20	Simulation Exercise	Pending
Electronics	2026-10-21	Sensor Circuit Exercise	Approve
Measurement Laboratory	2026-10-24	Measurement Exercise	Pending

One student index joins to three allocations across Computer, Electronics and Measurement Laboratory.

Reproduce: create an empty database, execute create_tables.sql, then sample_data.sql, then 05_student_history.sql. Query outputs describe the demonstration data, not live university usage.